

EIN092B: Laboratorio Técnicas de Visualización Exploratoria y Análisis de Textos

Guía de Actividades Sesión 6

22 al 26 de septiembre de 2025

1. Requisitos de entrada.

- Haber **entendido** las actividades de todas las sesiones anteriores.
- Contar con un repositorio personal de códigos diseñados de las sesiones previas.

2. Objetivos

- Procesar archivos de texto para extraer información relevante, utilizando técnicas de preprocesamiento de lenguaje natural.
- Construir representaciones vectoriales de los textos mediante técnicas especializadas, con el fin de obtener una representación estructurada que permita el análisis y manipulación de los datos.
- Realizar un análisis exploratorio de los datos de texto, visualizando patrones y tendencias utilizando gráficos estudiados en las sesiones anteriores.

3. Indicaciones generales.

Las actividades planteadas en esta guía deben ser entregadas durante la sesión de laboratorio.

3.1. Sobre el proceso de revisión de actividades.

Para el proceso de revisión consideren lo siguiente:

- La revisión y evaluación de cada una de las actividades se realizará en dos etapas: (i) una validación básica de requerimientos funcionales mínimos durante la sesión, y (ii) una verificación de requerimientos funcionales y no funcionales posterior a la sesión.
- Durante la sesión, avise al staff (profesor o ayudante) cuando complete una o varias actividades para que muestre y explique sus resultados, y responda las preguntas que se le realizarán sobre los conceptos asociados.

- Cualquier integrante del grupo debe ser capaz de responder las preguntas. Si las respuestas no son satisfactorias, o algún integrante no es capaz de responder, no se revisará la actividad hasta que demuestren que **TODO**s los integrantes del grupo demuestren un entendimiento mínimo sobre lo que están entregando.
- Durante la sesión, el staff marcará en una planilla cuando Ud. considere una actividad terminada y haya pasado la interrogación. El tener la actividad marcada como revisada en la sesión no entrega puntaje, y solo asegura que cumple con requerimientos para que entregue sus archivos y pase a la siguiente etapa de revisión. El formato de entrega de sus archivos se indica en la sección 3.2.
- En el caso de las actividades no entregadas en la sección correspondiente contará como no entregada a tiempo y **deberá entregar la actividad corregida en la siguiente sesión, aplicando descuentos por atraso.**
- En el caso de que algún integrante del grupo no este presente cuando su grupo correspondiente presente la evaluación y su correspondiente interrogación, deberá presentarlo por su propia cuenta en las siguientes sesiones y sufriendo la penalización por atraso.

3.2. Indicaciones de formato para entrega.

La evaluación constará de dos partes. La primera parte consiste en una validación básica y preguntas sobre los códigos propuestos por el grupo. La segunda parte constará de una verificación de sus códigos y respuestas, la cual se realizará sobre los archivos que deberá entregar vía Aula USM, los cuales deben seguir estrictamente las especificaciones que se le indican a continuación:

1. Debe crear una carpeta con el nombre *GXX_Sesion6*, donde *XX* el número de grupo. Por ejemplo, la carpeta para el grupo 5 se llamaría *G5_Sesion6*. El número de su grupo fue definido en la sesión anterior.
2. Dentro de esta carpeta, debe incluirse un archivo de Jupyter Notebook que contenga el desarrollo de las actividades y las respuestas correspondientes.

Recordar que **será responsabilidad del grupo entregar de manera correcta los archivos**. Se permitirá el envío de los archivos hasta las **23:59 del 27 de septiembre**. Sin embargo, solo se revisarán las actividades marcadas como revisadas durante la sesión. Cualquier actividad adicional no será considerada en la revisión.

4. Introducción al Procesamiento de Lenguaje Natural

El procesamiento del lenguaje natural (PLN) es una rama de la *inteligencia artificial* que se centra en el estudio de la interacción entre las computadoras y el lenguaje humano. Su principal objetivo es permitir que los algoritmos, interpreten y generen texto para resolver tareas específicas, como traducción automática, análisis de sentimientos, clasificación de texto, entre otras aplicaciones.

Por lo general, la mayoría de los desarrollos en el área de PLN están orientados al idioma inglés. Esto se debe a que el español presenta varios desafíos para los algoritmos de procesamiento del lenguaje natural debido a sus características lingüísticas. Algunos de estos desafíos están relacionados con la gran cantidad de formas verbales en español, derivadas de los distintos tiempos, modos y personas. Además, las palabras pueden tener diferentes significados según el contexto o la región. Estas características hacen que el procesamiento del lenguaje natural en español sea más complejo¹, ya que los modelos deben tener en cuenta una mayor variabilidad y complejidad lingüística en comparación con el inglés.

En esta sesión de laboratorio, utilizaremos la biblioteca *Natural Language Toolkit* (NLTK)² y trabajaremos con un conjunto de datos público en inglés.

4.1. Análisis de Sentimientos

El análisis de sentimientos se refiere al proceso de extraer información sobre la actitud que una persona expresa en un medio o formato digital respecto a un tema o contexto de comunicación. Uno de los casos más estudiados es la determinación de la polaridad de un fragmento de texto, es decir, la clasificación de una evaluación escrita (*review*) en la que se expresa una opinión, ya sea como *positiva*, *negativa* o *neutral*. Este enfoque también se ha extendido a otros ámbitos, como el análisis de la polaridad en textos provenientes de redes sociales.

La complejidad de este problema radica en la naturaleza ambigua e informal del lenguaje utilizado por las personas, así como en el manejo de aspectos como negaciones, sarcasmos y abreviaciones dentro de una frase.

5. Actividades

En esta actividad, utilizaremos un conjunto de datos correspondiente a un análisis de sentimientos basados en mensajes en *Twitter*. Este conjunto de datos incluye mensajes y una entidad, donde la tarea consiste en juzgar el sentimiento del mensaje hacia dicha entidad. En este laboratorio, nos enfocaremos en el preprocesamiento de los datos y en la aplicación de gráficos estudiados en sesiones anteriores para explorar el dataset.

5.1. Cargar datos y descripción del conjunto de datos

Como primer paso, se solicita que descargue el conjunto de datos Kaggle Twitter Entity Sentiment Analysis. En el enlace, podrá encontrar una descripción del dataset. En esta sesión de laboratorio solo utilizaremos los datos que se encuentran en el archivo **twitter_training.csv**. En sesiones posteriores, se empleará el conjunto de validación.

A continuación, se proporciona un fragmento de código de referencia para cargar los datos.

```
1 import pandas as pd
2
3 ftr = open(path_data, "r", encoding="ISO-8859-1")
```

¹Existen bibliotecas como SpaCy que permiten el trabajo con lenguajes como el español

²Documentación y ejemplos <https://www.nltk.org/>

```

4 rows = [line.strip().split(",", 3) for line in ftr.readlines()[1:4001]]
5 ftr.close()
6
7 df_data = pd.DataFrame(rows, columns=['Tweet ID', 'Entity', 'Sentiment', 'Text'])
8 df_data['Sentiment'] = df_data['Sentiment'].str.strip().str.lower()
9 sentiment_mapping = {'positive': 1, 'negative': 0, 'neutral': 2, 'irrelevant': 3}
10 df_data['Sentiment'] = df_data['Sentiment'].replace(sentiment_mapping)
11
12 df_data['Text'] = df_data['Text'].str.replace(r'([.,!?\()])', r' \1 ', regex=True)
13 df_data['Text'] = df_data['Text'].str.replace(r'\s{2,}', ' ', regex=True)
14 df_data['Text'] = df_data['Text'].str.lower()
15 df_data['Text'] = df_data['Text'].str.strip()

```

Una vez que se han cargado los datos, es necesario realizar una descripción del dataset, donde se respondan preguntas como: ¿cuántas clases de texto tiene el conjunto de datos? Además, se espera que comprenda el código y pueda explicar qué hace cada línea.

Seguidamente, elabore una descripción del conjunto de datos, destacando aspectos como la longitud promedio de los textos, la cantidad de ejemplos por clase y cualquier otra característica que considere importante para dimensionar el problema al que se enfrenta.

Adicionalmente, se le solicita que realice los siguientes gráficos empleando la librería `seaborn`³:

- Genere un histograma que muestre la distribución de la longitud de los textos en el conjunto de datos. Esto nos ayudará a comprender cómo varían las longitudes de los mensajes y a identificar posibles tendencias o patrones en los datos.
- Crear un boxplot que represente la longitud de los textos agrupados por las diferentes clases de sentimiento. Este análisis nos permitirá visualizar las diferencias en la longitud de los mensajes entre las distintas clases y evaluar si existen variaciones significativas.

Como referencia, se suministra el código para calcular la longitud de caracteres en cada texto en el dataset.

```

1 df_data['Length'] = df_data['Text'].apply(len) # Longitud en caracteres

```

5.2. Procesamiento de texto

Realice un preprocesamiento a los textos contenidos en el conjunto de datos con el fin de normalizar su estructura. Como punto de partida, debe asegurarse de tener el recurso de lematización de NLTK descargado.

```

1 import nltk
2 nltk.download('punkt')
3 nltk.download('wordnet')
4 nltk.download('stopwords')

```

³<https://seaborn.pydata.org/>

5.2.1. Stop words - lemmatizer

Puede iniciar la actividad del siguiente código de ejemplo, donde se convierte el texto a minúsculas (*lower-casing*), se reducen las letras repetidas, se eliminan palabras sin significado como artículos, pronombres y preposiciones (*stop word removal* [1]), y además se aplica una técnica de *lemmatizer* para llevar las palabras a su tronco léxico[2].

```

1 import re, time
2 from nltk.corpus import stopwords
3 from nltk import WordNetLemmatizer, word_tokenize
4 def base_word(word):
5     wordlemmatizer = WordNetLemmatizer()
6     return wordlemmatizer.lemmatize(word)
7 def word_extractor(text):
8     commonwords = stopwords.words('english')
9     text = re.sub(r'([a-z])\1+', r'\1\1', text) #Sustituya varias letras por dos
10    words = ""
11    wordtokens = [ base_word(word.lower()) for word in word_tokenize(text) ]
12    for word in wordtokens:
13        if word not in commonwords: #borrar stopwords
14            words+=" "+word
15    return words

```

A continuación, se proporcionan ejemplos para que pruebe las funciones presentadas anteriormente. Aplique la función `word_extractor` al conjunto de datos que se cargó en la actividad anterior.

```

1 word_extractor("Liverpool dominated possession but failed to score")
2 word_extractor("Arsenal secured a crucial victory in the final minutes")
3 # try
4 data_train = [ word_extractor(text) for text in df_train ]

```

Comente la importancia de la etapa de preprocesamiento en el dominio del lenguaje natural y la relevancia de cada una de las etapas de este proceso. Específicamente, explique para qué se utilizan: `stopwords`, `WordNetLemmatizer` y `word_tokenize`.

5.2.2. Stop words - stemming

Cambie el preprocesamiento del conjunto de datos. Para ello, reemplace el preprocesamiento realizado en la sección 5.2.1. Aplicando la técnica de *stemming*⁴ en lugar de la *lemmatizer* para reducir las palabras a su tronco léxico. A continuación, se provee un código de referencia para aplicar la técnica *stemming*.

```

1 from nltk.stem.porter import PorterStemmer
2 wordstemmer = PorterStemmer()
3 wordstemmer.stem(word)

```

Compare las frases obtenidas al aplicar *stop words* con las técnicas *stemming* y *lemmatizer* ¿Que diferencias encuentra en los dos enfoques? Investigue sobre las ventajas y desventajas.

Finalmente, en algunos casos, es recomendable eliminar los símbolos y los números de las frases para conservar solo los caracteres alfabéticos. Para el conjunto de datos de esta sesión, ¿considera

⁴Información y ejemplos en <https://www.ibm.com/topics/stemming>

necesario eliminar tanto los símbolos como los números, o únicamente los símbolos?

```
1 # Eliminar todo lo que no sea letras o espacios.
2 text = re.sub(r'^a-zA-Z\s', '', text)
3 # Eliminar todo lo que no sea letras, n meros o espacios.
4 text = re.sub(r'^a-zA-Z0-9\s', '', text)
```

5.3. Embeddings y Representaciones Vectoriales

Un *embedding* es una representación vectorial densa de palabras, que captura sus relaciones semánticas y contextuales. Los embeddings permiten que palabras con significados similares tengan vectores cercanos en el espacio vectorial. Esto se logra a través de algoritmos que analizan grandes cantidades de texto y aprenden a mapear palabras a vectores de alta dimensión. A continuación, se explora la representación vectorial *term-frequency*.

5.3.1. Representación Vectorial *term-frequency*

Construya una representación vectorial de los textos de entrada con el fin de manipularlos y, posteriormente, puedan ser empleados para otro tipo de proceso, incluso para alimentar algoritmos de aprendizaje de máquinas. Para esto, utilice uno de los tipos de características más comunes, que consiste en contar cuántas veces aparece cada término o palabra en el texto, denominado *TF* (*term-frequency*). El vocabulario base se construirá a partir de la unión de todas las palabras que aparezcan en los textos de entrenamiento.

```
1 import numpy as np
2 from sklearn.feature_extraction.text import CountVectorizer
3 texts_train = df_data['Processed_Text'].tolist()
4
5 # Vectorización TF usando CountVectorizer
6 vectorizer = CountVectorizer(ngram_range=(1, 1), binary=False)
7 vectorizer.fit(texts_train)
8 features_train = vectorizer.transform(texts_train)
```

Actividades a realizar:

- La función `CountVectorizer` se utiliza para convertir un conjunto de textos en una matriz de características basada en la frecuencia de las palabras. Describa los argumentos de la función `CountVectorizer` y sus argumentos internos. ¿Qué sucedería si `ngram_range` se configura con otros valores, por ejemplo `ngram_range = (2 × 2)`? Pruebe con distintas configuraciones y verifique el tamaño del conjunto de entrenamiento resultante.
- Asimismo, discuta sobre `features_train`. ¿Podrían usarse estas características para entrenar un modelo de aprendizaje automático? Si inspecciona `features_train`, notará que es una matriz *sparse* (dispersa). Explique por qué se usan ese tipo de matrices.
- Seguidamente, use la función `get_feature_names_out()` para extraer todas las palabras presentes en los textos. ¿Cuántas palabras tiene el dataset?

```
1 # Transformar y obtener vocabulario
2 vocab = vectorizer.get_feature_names_out()
```

```

3
4 # Sumatoria de las frecuencias de las palabras en todos los textos
5 dist = list(np.array(features_train.sum(axis=0)).reshape(-1,))
6
7 word_freq = sorted(list(zip(vocab, dist)), key=lambda x: x[1], reverse=True)

```

- Elabore dos gráficos de barras con las 30 palabras más frecuentes en el conjunto de datos. Considere una distribución de 1×2 para las filas y columnas. A la derecha, incluya el gráfico con los datos preprocesados usando *lemmatizer*, y a la izquierda, el gráfico con los datos preprocesados usando *stemming*.
- Por último, genere un gráfico de nube de palabras empleando la librería *WordCloud*. Tenga en cuenta que este tipo de gráfico se puede visualizar utilizando `plt.imshow()`.

```

1 from wordcloud import WordCloud
2 import matplotlib.pyplot as plt
3
4 word_freq_2 = dict(zip(vocab, dist))
5
6 # Generar la WordCloud
7 wordcloud = WordCloud(width=800, height=400, background_color='white').
    generate_from_frequencies(word_freq_2)
8

```

Otras técnicas de representación vectorial más avanzadas que *TF* son *Word2Vec* y *GloVe*, las cuales pertenecen a un tema más avanzado que se incentiva a explorar. Sin embargo, se requieren conocimientos de cursos posteriores, dado que estas representaciones vectoriales emplean técnicas de aprendizaje automático para generar embeddings.

Referencias

- [1] A. Rajaraman y J. D. Ullman. “Data Mining”. En: *Mining of Massive Datasets*. Cambridge University Press, 2011, págs. 1-17.
- [2] Thomas Müller et al. “Joint Lemmatization and Morphological Tagging with Lemming”. En: *ArXiv abs/2405.18308* (2015). URL: <https://api.semanticscholar.org/CorpusID:4248450>.